

OP2

Generated by Doxygen 1.9.4

1 Paleidimo instrukcijos	1
1.1 v3.0 Aprašymas	1
1.1.1 Sukurtas .exe failas instaliuoti programą Windows operacinėse sistemose	1
1.1.1.1 Instaliavimo instrukcijos	1
1.1.2 Sukurta Vector klasė Student1 klasės objektų saugojimui	2
1.1.2.1 Vector klasės bazinės funkcijos ir jų aprašymai	2
1.1.2.2 Papildyti unit testai dabar testuoja ir naująją Vector klasę	2
1.1.3 Atlikta spartumo analizė (Vector klasė vs std::vector)	2
1.2 v2.0 Aprašymas	3
1.2.1 Sukurti unit testai naudojant Catch2	3
1.2.1.1 Kas yra tikrinama unit testuose?	3
1.2.1.2 Kaip paleisti unit testus	3
1.3 Randomness seeded to: 2873484529	4
1.3.1 Setters	4
1.3.2 Move Constructor	4
1.3.3 Reading from file	4
1.3.3.1 Dokumentacijos generavimas	4
1.3.3.2 Apibendrinimas	5
1.4 v1.1 Tyrimas	5
1.4.1 Pirma strategija	6
1.5 Vektorius	6
1.6 Listas	7
1.7 Dekas	8
1.7.1 Pirmosios strategijos išvados	9
1.7.2 Antra strategija	9
1.8 Vektorius	9
1.9 Listas	10
1.10 Dekas	11
1.10.1 Antrosios strategijos išvados	12
1.10.2 Trečia strategija	13
1.10.3 Trečiosios strategijos išvados	13
1.10.4 Visų strategijų išvados	14
1.10.5 Kompiuterio komponentai	14
2 Hierarchical Index	17
2.1 Class Hierarchy	17
3 Class Index	19
3.1 Class List	19
4 File Index	21
4.1 File List	21
5 Class Documentation	23

5.1 Human Class Reference	23
5.2 Student1 Class Reference	24
5.2.1 Member Function Documentation	25
5.2.1.1 getPavarde()	25
5.2.1.2 getVardas()	25
5.3 Vector< T > Class Template Reference	25
6 File Documentation	27
6.1 header.h	27
6.2 human.hpp	27
6.3 student.hpp	28
6.4 vector.hpp	31
Index	37

Chapter 1

Paleidimo instrukcijos

- Klonuojame arba parsisiunčiame šitą repozitoriją
- Šio projekto direktorijoje atsidarome terminalą ir rašome `make`
- Turi atsirasti aplankas: `obj`
- Paleidžiame progamą naudodami komandą `./main`
- Baigus darbą ištriname visus komandos `make` sugeneruotus failus naudodami komandą `make clean`
- Kilus klausimams ar nesusipratimams, prašome sukurti naują issue github sistemoje

1.1 v3.0 Aprašymas

1.1.1 Sukurtas .exe failas instaliuoti programą Windows operacinėse sistemose

1.1.1.1 Instaliavimo instrukcijos

- Parsisiunčiame `Setup_BenediktasJuozapaitis.exe` iš šios repozitorijos
- Parsisiuntus failui, spaudžiame ant jo
- Suteikiame administratoriaus privilegijas
- Visur spaudžiame mygtuką "next"
- Programa turėtų automatiškai susinstaliuoti "`C:\Program Files (x86)\VU\Benediktas-Juozapaitis`"
- Programą galima paleisti ant ikonėles du kartus paspaudus darbalaukyje arba meniu juostoje `VU->Benediktas Juozapaitis`

1.1.2 Sukurta Vector klasė Student1 klasės objektų saugojimui

1.1.2.1 Vector klasės bazinės funkcijos ir jų aprašymai

- **Konstruktorius**

```
Vector() : data(nullptr), size(0), capacity(0) {}
```

- **Move konstruktorius**

```
Vector(Vector&& other) noexcept
: data(other.data), size(other.size), capacity(other.capacity) {
    other.data = nullptr;
    other.size = 0;
    other.capacity = 0;
}
```

Duomenys iš kito vektoriaus perkeliama į dabartinį vektorių ir duomenys yra pašalinami (other.data = nullptr)

- **Destruktorius**

```
~Vector() {
    delete[] data;
}
```

Ištrinamas masyvas - atlaisvinama nenaudojama vieta

- **operator[]**

```
T& operator[](size_t index) {
    return data[index];
}
```

Leidžia pasiekti elementą naudojant [] operatorių

- ****at()****

```
T& at(size_t index) {
    if (index >= size) {
        throw std::out_of_range("Index out of range");
    }
    return data[index];
}
```

Leidžia pasiekti elementą taip pat kaip ir **operator[]**, tačiau dar patikrina ar vartotojo indeksas teisingas. Jei yra neteisingas, vartotojas yra informuojamas pranešimu

1.1.2.2 Papildyti unit testai dabar testuoja ir naująją Vector klasę

Testus galite rasti: `unit_tests/main.cpp` Naudojimosi instrukcija yra aprašyta **V2.0** aprašyme

1.1.3 Atlikta spartumo analizė (Vector klasė vs std::vector)

- **Analizės metodas** Lyginama kiek vidutiniškai laiko užtrunka laiko užpildyti `std::vector` ir nuosavo **Vector** tipo tuščius vektorius : 10000, 100000, 1000000, 10000000 ir 100000000 int tipo elementų naudojant `push_back()` funkciją
- **Duomenys (average in milliseconds)**

Container type	10000	100000	1000000	10000000	100000000
<code>std::vector</code>	0	7	41	202	2055
Vector	0	3	14	110	1388
Difference	0	4	27	92	667

****Išvadaos**** Savadarbis **Vector** pagrindu sukurtas konteineris yra beveik dvigubai efektyvesnis kai yra naudojama `push_back()` funkcija lyginant su vektorium iš STL bibliotekos

- **Analizės metodas** Lyginama kiek vidutiniškai laiko užtrunka laiko užpildyti `std::vector` ir nuosavo `Vector` tipo tuščius vektorius : 1000000, 10000000 ir 100000000 `Student1` elementų
- **Duomenys (average in seconds)**

Container type	1000000	10000000	100000000
<code>std::vector</code>	0.57	5.15	54.86
<code>Vector</code>	1.15	13.4	168
Difference	-0.58	-8.65	-113.14

- **Išvadaos** Savadarbis `Vector` pagrindu sukurtas konteineris yra neefektyvus saugant kompleksines duomenų struktūras lyginant su vektorium iš STL bibliotekos

1.2 v2.0 Aprašymas

1.2.1 Sukurti unit testai naudojant [Catch2](https://github.com/catchorg/Catch2)

1.2.1.1 Kas yra tikrinama unit testuose?

Unit testuose yra tikrinamas klasės `Student1` funkcionalumas ir veikimas.

Specifiškai yra tikrinami:

- Setter'iai
- Default konstruktorius
- Copy konstruktorius
- Copy assignment operatorius
- Move konstruktorius
- Move assignment operatorius
- Objektų skaitymas iš failo
- Objektų saugojimas `std::vector` konteineryje

1.2.1.2 Kaip paleisti unit testus

- Pirmiausia reikia nueiti į aplanką `unit_tests`. Tai galima padaryti naudojant \
 - + Sukompiliuoti jame esančius failus. Tam galima naudoti komandą \

```
``g++ *.cpp -o unit_tests
```
- Tada paleidžiame sukompiliuotus unit testus naudodami komandą \

```
{/unit_tests``}
```

 - + Jei testai buvo įvykdyti **sėkmingai**, po programos paleidimo turėtumėte matyti šį tekstą:

1.3 Randomness seeded to: 2873484529

All tests passed (34 assertions in 8 test cases)

* "Randomness seeded to:" skaičius gali skirtis
+ Jei testai buvo įvykdyti **nesėkmingai**, po programos paleidimo turėtumėte matyti šį tekstą:

Randomness seeded to: 1317810014

~~~~~  
unit\_tests is a Catch2 v3.8.1 host application. Run with -? for options

### 1.3.1 Setters

main.cpp:4 .....

main.cpp:12: FAILED: REQUIRE( studentas.getVardas() == "Admas" ) with expansion: "Adomas" == "Admas"

### 1.3.2 Move Constructor

main.cpp:42 .....

main.cpp:47: FAILED: REQUIRE( b.getVardas() == "Gente" ) with expansion: "Genute" == "Gente"

### 1.3.3 Reading from file

main.cpp:70 .....

main.cpp:77: FAILED: REQUIRE( studentas.getPavarde() == "Pavrde0" ) with expansion: "Pavarde0" == "Pavrde0"

test cases: 8 | 5 passed | 3 failed assertions: 25 | 22 passed | 3 failed

Programos išvestis parodo, ko test case'as tikėjosi, ir kokią reikšmę gavo. T.y. parodo kur testai įvyko nesėkmingai.\

Išvesties apačioje galima matyti bendrą statistiką - kiek testų įvyko sėkmingai ir kiek nesėkmingai.

## Sukurta dokumentacija naudojant [Doxygen] (<https://www.doxygen.nl/index.html>)

### Kaip naudotis šia dokumentacija?

Dokumentaciją galima peržiūrėti dviejais būdais:

+ Kaip ``.html`` failą

+ Kaip ``.pdf`` failą

+ Norint dokumentaciją peržiūrėti kaip ``.html`` failą, jums reikės jį atsidaryti bet kurioje naršyklėje.\

Šis failas yra šioje lokacijoje: ``/documentation/html/index.html``

- Norint dokumentaciją peržiūrėti kaip .pdf failą, jums reikės jį atsidaryti bet kurioje .pdf failus skaitančioje programoje.\ Šis failas yra šioje lokacijoje: /documentation/latex/refman.pdf

#### 1.3.3.1 Dokumentacijos generavimas

1. Pirmiausia reikia turėti šiuos įrankius:

- doxygen
- graphviz
- texlive texlive-latex-extra texlive-fonts-recommended
- make

1. Reikia nueiti į direktoriją documentation

2. Tada terminale parašome komandą:

```
4. Turi atsirasti du aplankai:
+ ``html
```

- latex

1. Nueiname į latex direktoriją ir parašome terminale komandą:

```
{make``}
```

Tada sekdami žingsnius nurodytus anksčiau galima bus atsidaryti dokumentaciją norimu formatu

```
# v1.5 Aprašymas
```

```
## Human klasė
```

Sukurta bazinę (abstrakti) klasę skirta bendrai aprašyti žmogų.\

Iš jos išeina (dervied) klasė Student1.

```
# v1.2 Aprašymas
```

```
## Implementuota "Rule of Five" Studentų klasei
```



```

| Funkcijos pavadinimas | Funkcijos implementacija |
|-----|-----|
|Destruktorius|[] (images/destruktorius.png)|
|Copy konstruktorius|[] (images/copy%20konstruktorius.png)|
|Copy assignmentas|[] (images/copy%20assignmentas.png)|
|Move konstruktorius|[] (images/move%20konstruktorius.png)|
|Move assignmentas|[] (images/move%20assignmentas.png)|
## Perdengti įvesties ir išvesties metodai
|||
|-|-|
|Ivestis|[] (images/operator».png)|
|Išvestis|[] (images/operator«.png)|
## **Klasės Student1** Objektų įvestis / išvestis
### Įvestis
+ Obejktus galima įvesti naudotj at įvesties operatorių `»`:

```

**Student1** studentas; // inicializuojame studentą su defaultinėmis reikšmėmis std::cin >> studentas; // užpildome norimomis reikšmėmis

+ Taip pat galima įvesti duomenis į objektus juos inicializuojant:

**Student1** studentas ("Vardenis", "Pavardenis", 10, {9,8,7});

+ Galima naudoti Setterius:

**Student1** studentas; // inicializuojame studentą su defaultinėmis reikšmėmis // priskiriame reikšmes  
: studentas.SetVardas("Tomas"); studentas.SetPavarde("Tomaitis"); studentas.SetEgzaminas(10);  
studentas.SetMark({1,2,3,4});  
### Išvestis  
+ Obejktus galima išvesti naudotj at įvesties operatorių `«`:

// Išvedimas į ekraną

**Student1** studentas; // inicializuojame studentą su defaultinėmis reikšmėmis std::cout << studentas; // kadangi objektui nebuvo priskirta jokių reikšmių, bus išvestos defaultinės reikšmės

// Išvedimas į failą

**Student1** studentas; // inicializuojame studentą su defaultinėmis reikšmėmis std::ofstream out ("output.txt");  
out << studentas; // kadangi objektui nebuvo priskirta jokių reikšmių, bus išvestos defaultinės reikšmės  
+ Objektus galima taip pat išvesti / gauti naudojant Getterius:

**Student1** studentas ("Vardenis", "Pavardenis", 10, {9,8,7}); // sukuriame objektą ir priskiriame jam reikšmes  
std::string vardas = studentas.GetVardas() // result: Vardenis std::string pavarde = studentas.GetPavarde()  
// result: Pavardenis int egzaminas = studentas.GetEgzaminas() // result: 10 std::vector<int> mark =  
studentas.GetMark() // result: {9,8,7} ``

### 1.3.3.2 Apibendrinimas

Duomenis apie objektus galima labai paprastai išvesti tiek į failą tiek į ekraną/terminalą naudojant operatorių <<, arba getterius, norint failus nuskaityti iš failo, naudojami setteriai, jeigu duomenys generuojami pačios programos, taip pat yra naudojami setteriai, tačiau jei vartotojas nori įvesti duomenis programos eigoje galima naudoti įvesties operatorių >>.

## 1.4 v1.1 Tyrimas

### Struct vs class (vector)

| Implementacijos tipas | Laikas 1000000.txt | Laikas 10000000.txt |
|-----------------------|--------------------|---------------------|
| Class                 | 9.04668            | 102.168             |
| Struct                | 5.157186           | 54.86644            |

### Optimizacijos veliaveles

| Veliavele | Laikas (s) | Failo dydis (kB) |
|-----------|------------|------------------|
| None      | 100        | 262.5            |
| O1        | 25.34      | 123.4            |
| O2        | 24.64      | 145.6            |
| O3        | 23.54      | 147.6            |

### 1.4.1 Pirma strategija

Bendro studentai konteinerio (vector, list ir deque tipų) skaidymas (rūšiavimas) į du naujus to paties tipo konteinerius: "vargšiukų" ir "kietiukų". Tokiu būdu tas pats studentas yra dvejuose konteineriuose: bendrame studentai ir viename iš suskaidytų (vargšiukai arba kietiakai). Nesunku pastebėti, kad tokia strategija yra neefektyvi užimamos atminties atžvilgiu (įsitikinkite tuo!), tačiau šiame žingsnyje svarbiausia yra patyrinėti, kaip programos veikimo sparta priklauso nuo konteinerio tipo?

## 1.5 Vektorius

1000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.0157612   | 0.00297164 | 0.00109384               | 0.035876       |
| 2               | 0.00500573  | 0.00170572 | 0.0869301                | 0.0991567      |
| 3               | 0.00541565  | 0.00165097 | 0.0409119                | 0.0534136      |
| 4               | 0.00515032  | 0.00174594 | 0.0447787                | 0.0571739      |
| 5               | 0.00514549  | 0.00164459 | 0.042081                 | 0.0553084      |
| Vid.            | -           | -          | -                        | 0.06018572     |

10000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.0465477   | 0.0218913  | 0.00751638               | 0.0835118      |
| 2               | 0.0387483   | 0.0188099  | 0.00303522               | 0.103406       |
| 3               | 0.0505536   | 0.0180376  | 0.00315221               | 0.104323       |
| 4               | 0.0509217   | 0.0185617  | 0.00295333               | 0.107807       |
| 5               | 0.0442961   | 0.0196937  | 0.00486322               | 0.101467       |
| Vid.            | -           | -          | -                        | 0.10010296     |

100000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.294555    | 0.205679   | 0.0498312                | 0.555141       |
| 2               | 0.248458    | 0.207416   | 0.0480726                | 0.509017       |
| 3               | 0.250679    | 0.206326   | 0.0411719                | 0.503209       |
| 4               | 0.258195    | 0.206838   | 0.0394575                | 0.512261       |
| 5               | 0.242133    | 0.20773    | 0.040613                 | 0.495552       |
| Vid.            | -           | -          | -                        | 0.515036       |

1000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 2.01091     | 3.13237    | 0.592756                 | 5.74805        |
| 2               | 2.03059     | 2.76148    | 0.377035                 | 5.1739         |
| 3               | 1.96574     | 2.76007    | 0.368503                 | 5.09911        |
| 4               | 1.95757     | 2.75842    | 0.364877                 | 5.08575        |
| 5               | 1.95784     | 2.78063    | 0.369691                 | 5.11322        |
| Vid.            | -           | -          | -                        | 5.244066       |

10000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 21.268      | 31.3564    | 4.9157                   | 57.5453        |
| 2               | 18.6642     | 31.192     | 4.2402                   | 54.1014        |

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 3               | 18.5695     | 31.0392    | 4.27658                  | 53.8901        |
| 4               | 18.5498     | 31.0862    | 4.27511                  | 53.9161        |
| 5               | 18.646      | 31.1166    | 4.20896                  | 53.9763        |
| Vid.            | -           | -          | -                        | 54.68584       |

## 1.6 Listas

1000

| Bandymo numeris | Nuskaitymas | Rūšiavimas  | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|-------------|--------------------------|----------------|
| 1               | 0.028673    | 0.00224555  | 0.0065434                | 0.0485917      |
| 2               | 0.0141865   | 0.000592377 | 0.00445487               | 0.0398903      |
| 3               | 0.0127271   | 0.000681703 | 0.00467639               | 0.0307993      |
| 4               | 0.015389    | 0.000613232 | 0.00423966               | 0.0390246      |
| 5               | 0.010633    | 0.00124906  | 0.00620234               | 0.0344741      |
| Vid.            | -           | -           | -                        | 0.038556       |

10000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.0755908   | 0.00776333 | 0.0379893                | 0.12623        |
| 2               | 0.112248    | 0.00799475 | 0.0344115                | 0.16415        |
| 3               | 0.0982861   | 0.00723372 | 0.0412163                | 0.160044       |
| 4               | 0.108907    | 0.00826699 | 0.0383505                | 0.168245       |
| 5               | 0.134019    | 0.00846876 | 0.0349826                | 0.19223        |
| Vid.            | -           | -          | -                        | 0.1621798      |

100000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.396517    | 0.132524   | 0.215003                 | 0.74913        |
| 2               | 0.750414    | 0.113464   | 0.216181                 | 1.08489        |
| 3               | 0.735554    | 0.146832   | 0.218128                 | 1.10574        |
| 4               | 0.67686     | 0.123267   | 0.224174                 | 1.03264        |
| 5               | 0.721461    | 0.137419   | 0.218994                 | 1.08686        |
| Vid.            | -           | -          | -                        | 1.01146        |

1000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 3.33569     | 1.49648    | 1.81773                  | 6.65532        |
| 2               | 3.94941     | 1.52792    | 1.82485                  | 7.3076         |
| 3               | 3.50662     | 1.4599     | 1.80844                  | 6.78118        |
| 4               | 3.59917     | 1.45591    | 1.83417                  | 6.89438        |
| 5               | 3.53286     | 1.45816    | 1.80034                  | 6.79671        |
| Vid.            | -           | -          | -                        | 6.887438       |

10000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 38.3839     | 17.0501    | 30.2436                  | 85.7153        |
| 2               | 36.2216     | 16.6287    | 46.4983                  | 99.3648        |
| 3               | 36.2565     | 16.5871    | 36.9609                  | 89.8366        |

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 4               | 36.2117     | 16.7599    | 39.9306                  | 92.9319        |
| 5               | 36.1921     | 16.6807    | 41.8136                  | 94.7047        |
| Vid.            | -           | -          | -                        | 92.91066       |

## 1.7 Dekas

1000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.0284543   | 0.00478177 | 0.00666952               | 0.0683668      |
| 2               | 0.0217629   | 0.00367485 | 0.00142868               | 0.0324457      |
| 3               | 0.0397455   | 0.00363872 | 0.0116896                | 0.0629371      |
| 4               | 0.02948     | 0.00360505 | 0.0126408                | 0.0526156      |
| 5               | 0.0415463   | 0.0042363  | 0.0109747                | 0.063883       |
| Vid.            | -           | -          | -                        | 0.05684944     |

10000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.148513    | 0.0358707  | 0.0138857                | 0.204631       |
| 2               | 0.100335    | 0.0327386  | 0.0161345                | 0.154735       |
| 3               | 0.230075    | 0.0421036  | 0.16088                  | 0.438552       |
| 4               | 0.341478    | 0.0379575  | 0.181016                 | 0.566294       |
| 5               | 0.232374    | 0.0413409  | 0.12454                  | 0.404222       |
| Vid.            | -           | -          | -                        | 0.3532868      |

100000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.551       | 0.406492   | 0.123125                 | 1.08569        |
| 2               | 0.55789     | 0.410391   | 0.121526                 | 1.09505        |
| 3               | 1.40657     | 0.608874   | 0.979027                 | 3.00015        |
| 4               | 1.59498     | 0.606944   | 1.33788                  | 3.546          |
| 5               | 1.55368     | 0.648659   | 1.17013                  | 3.3781         |
| Vid.            | -           | -          | -                        | 2.421398       |

1000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 4.72384     | 5.48063    | 1.39846                  | 11.6081        |
| 2               | 4.90894     | 5.5603     | 1.43335                  | 11.9075        |
| 3               | 11.6045     | 5.46964    | 11.9079                  | 28.9872        |
| 4               | 13.4975     | 5.48779    | 11.3384                  | 30.3287        |
| 5               | 13.4702     | 5.65318    | 10.7591                  | 29.8875        |
| Vid.            | -           | -          | -                        | 22.9448        |

10000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 169.44      | 190.127    | 931.583                  | 1291.33        |
| 2               | 281.898     | 222.961    | 1434.5                   | 1940.28        |
| 3               | 426.143     | 256.178    | 1347.28                  | 2029.72        |
| 4               | 326.435     | 218.653    | 1228.2                   | 1773.56        |

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 5               | 367.112     | 218.5      | 1334.59                  | 1920.86        |
| Vid.            | -           | -          | -                        | 1791.95        |

### 1.7.1 Pirmosios strategijos išvados

1000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 0.06018572             |
| Listas            | 0.038556               |
| Dekas             | 0.05684944             |

10000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 0.10010296             |
| Listas            | 0.1621798              |
| Dekas             | 0.3532868              |

100000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 0.515036               |
| Listas            | 1.01146                |
| Dekas             | 2.421398               |

1000000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 5.244066               |
| Listas            | 6.887438               |
| Dekas             | 22.9448                |

10000000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 54.68584               |
| Listas            | 92.91066               |
| Dekas             | 1791.95                |

### 1.7.2 Antra strategija

Bendro studentų konteinerio (vector, list ir deque) skaidymas (rūšiavimas) panaudojant tik vieną naują konteinerį: "vargšiukai". Tokiu būdu, jei studentas yra vargšiukas, jį turime įkelti į naująjį "vargšiukų" konteinerį ir ištrinti iš bendro studentai konteinerio. Po šio žingsnio studentai konteineryje liks vien tik kietiakai. Atminties atveju tai efektyviau, tačiau dažni trynimai gali būti "skausmingi", ypač tam tikro tipo konteineriams.

## 1.8 Vektorius

1000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.0224947   | 0.00333382 | 0.000640099              | 0.0353559      |

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 2               | 0.0188831   | 0.00490042 | 0.000791805              | 0.0364902      |
| 3               | 0.0153758   | 0.006616   | 0.0674435                | 0.105337       |
| 4               | 0.0115664   | 0.00495628 | 0.00162373               | 0.0324051      |
| 5               | 0.0176274   | 0.00496394 | 0.00389951               | 0.0409249      |
| Vid.            | -           | -          | -                        | 0.05090222     |

## 10000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.0435087   | 0.018539   | 0.00678181               | 0.0759039      |
| 2               | 0.0521218   | 0.0155541  | 0.00347023               | 0.0763377      |
| 3               | 0.0429066   | 0.0185378  | 0.00252914               | 0.068913       |
| 4               | 0.0582204   | 0.0198764  | 0.00720329               | 0.0923209      |
| 5               | 0.0540278   | 0.0183224  | 0.00711938               | 0.0852504      |
| Vid.            | -           | -          | -                        | 0.07974518     |

## 100000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.235309    | 0.214016   | 0.0395527                | 0.496738       |
| 2               | 0.229319    | 0.210091   | 0.039566                 | 0.483699       |
| 3               | 0.224111    | 0.212939   | 0.0354294                | 0.477368       |
| 4               | 0.228269    | 0.209659   | 0.0376307                | 0.480509       |
| 5               | 0.225       | 0.209522   | 0.0317681                | 0.471047       |
| Vid.            | -           | -          | -                        | 0.4810722      |

## 1000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 1.84954     | 2.71316    | 0.294114                 | 4.86164        |
| 2               | 1.94441     | 2.93612    | 0.321887                 | 5.20739        |
| 3               | 1.85822     | 2.88551    | 0.307638                 | 5.05856        |
| 4               | 1.70701     | 2.72363    | 0.288937                 | 4.72445        |
| 5               | 1.68065     | 2.68732    | 0.289074                 | 4.66199        |
| Vid.            | -           | -          | -                        | 4.902806       |

## 10000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 21.1653     | 32.1014    | 3.52579                  | 56.7974        |
| 2               | 21.913      | 38.631     | 4.05256                  | 64.6025        |
| 3               | 20.9693     | 32.4221    | 3.67551                  | 57.0718        |
| 4               | 20.2851     | 32.0449    | 3.11768                  | 55.4525        |
| 5               | 20.9391     | 32.0266    | 3.07497                  | 56.0454        |
| Vid.            | -           | -          | -                        | 57.99392       |

## 1.9 Listas

## 1000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.0246861   | 0.00162019 | 0.00176294               | 0.0411177      |
| 2               | 0.0308339   | 0.00167676 | 0.00340816               | 0.0539312      |

| Bandymo numeris | Nuskaitymas | Rūšiavimas  | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|-------------|--------------------------|----------------|
| 3               | 0.218818    | 0.000461914 | 0.000941641              | 0.224631       |
| 4               | 0.235189    | 0.000424114 | 0.00104226               | 0.24108        |
| 5               | 0.00694151  | 0.000404591 | 0.000952493              | 0.187131       |
| Vid.            | -           | -           | -                        | 0.1495786      |

## 10000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.0890261   | 0.00774911 | 0.0122458                | 0.114153       |
| 2               | 0.113501    | 0.0067795  | 0.0141554                | 0.139493       |
| 3               | 0.0854599   | 0.00712458 | 0.0141996                | 0.111854       |
| 4               | 0.0774984   | 0.00736629 | 0.011981                 | 0.101658       |
| 5               | 0.0760125   | 0.00714243 | 0.0118847                | 0.100492       |
| Vid.            | -           | -          | -                        | 0.1139308      |

## 100000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.402168    | 0.102229   | 0.0701497                | 0.579658       |
| 2               | 0.403838    | 0.103182   | 0.0752006                | 0.587199       |
| 3               | 0.417029    | 0.103555   | 0.0720878                | 0.597918       |
| 4               | 0.392518    | 0.111442   | 0.0726875                | 0.581787       |
| 5               | 0.409746    | 0.101969   | 0.0711279                | 0.587922       |
| Vid.            | -           | -          | -                        | 0.5868968      |

## 1000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 3.54817     | 1.7907     | 0.671718                 | 6.01564        |
| 2               | 3.73584     | 1.70472    | 0.506568                 | 5.95222        |
| 3               | 3.87421     | 2.01056    | 0.686919                 | 6.57708        |
| 4               | 3.70923     | 1.95394    | 0.629358                 | 6.29851        |
| 5               | 3.46178     | 1.71935    | 0.487398                 | 5.67413        |
| Vid.            | -           | -          | -                        | 6.103116       |

## 10000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 41.3789     | 22.1313    | 6.00292                  | 69.5215        |
| 2               | 41.1692     | 25.2102    | 5.8939                   | 72.2804        |
| 3               | 44.8078     | 23.7313    | 5.77993                  | 74.3261        |
| 4               | 40.9516     | 18.5867    | 7.20607                  | 66.7555        |
| 5               | 37.5642     | 16.8478    | 5.33181                  | 59.7492        |
| Vid.            | -           | -          | -                        | 68.92654       |

## 1.10 Dekas

## 1000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į konteinerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.0298667   | 0.00483818 | 0.000675241              | 0.0436806      |
| 2               | 0.0286584   | 0.00367079 | 0.000706936              | 0.0458477      |
| 3               | 0.0381032   | 0.00310144 | 0.000646202              | 0.0540752      |

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į kontainerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 4               | 0.0334661   | 0.00399885 | 0.000914481              | 0.0472957      |
| 5               | 0.037506    | 0.00519079 | 0.000790669              | 0.0529864      |
| Vid.            | -           | -          | -                        | 0.04837712     |

10000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į kontainerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.127815    | 0.0432774  | 0.00428575               | 0.186936       |
| 2               | 0.193597    | 0.0390235  | 0.00530735               | 0.242876       |
| 3               | 0.209365    | 0.0338897  | 0.00878288               | 0.262004       |
| 4               | 0.0938933   | 0.0345508  | 0.00622441               | 0.139556       |
| 5               | 0.282808    | 0.0361259  | 0.0048835                | 0.329362       |
| Vid.            | -           | -          | -                        | 0.2323468      |

100000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į kontainerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 0.566389    | 0.605203   | 0.0671125                | 1.2517         |
| 2               | 1.19945     | 0.433904   | 0.0548006                | 1.693          |
| 3               | 1.04132     | 0.405345   | 0.0509681                | 1.50256        |
| 4               | 0.544427    | 0.408635   | 0.0501242                | 1.00812        |
| 5               | 0.994666    | 0.39916    | 0.0515276                | 1.45062        |
| Vid.            | -           | -          | -                        | 1.3812         |

1000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į kontainerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 4.70215     | 6.3279     | 0.453336                 | 11.4895        |
| 2               | 4.42072     | 6.3346     | 0.463191                 | 11.2232        |
| 3               | 4.92799     | 5.38096    | 0.454459                 | 10.7683        |
| 4               | 4.35941     | 5.22606    | 0.443794                 | 10.0342        |
| 5               | 4.72555     | 5.23286    | 0.435385                 | 10.3991        |
| Vid.            | -           | -          | -                        | 10.78286       |

10000000

| Bandymo numeris | Nuskaitymas | Rūšiavimas | Skrstymas į kontainerius | Bendras laikas |
|-----------------|-------------|------------|--------------------------|----------------|
| 1               | 215.697     | 392.367    | 415.62                   | 1023.74        |
| 2               | 3067.47     | 456.818    | 652.337                  | 4177.23        |
| 3               | 213.666     | 390.526    | 335.602                  | 939.9          |
| 4               | 216.16      | 229.551    | 187.215                  | 633.343        |
| 5               | 177.097     | 219.953    | 178.219                  | 575.358        |
| Vid.            | -           | -          | -                        | 1270.3142      |

### 1.10.1 Antrosios strategijos išvados

1000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 0.05090222             |
| Listas            | 0.1495786              |
| Dekas             | 0.04837712             |

10000



| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 0.07974518             |
| Listas            | 0.1139308              |
| Dekas             | 0.2323468              |

100000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 0.4810722              |
| Listas            | 0.5868968              |
| Dekas             | 1.3812                 |

1000000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 4.902806               |
| Listas            | 6.103116               |
| Dekas             | 10.78286               |

10000000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 57.99392               |
| Listas            | 68.92654               |
| Dekas             | 1270.3142              |

### 1.10.2 Trečia strategija

*Bendro studentų konteinerio (vector, list ir deque) skaidymas (rūšiavimas) panaudojant greičiausiai veikianti 1 arba 2 strategiją įtraukiant į ją "efektyvius" darbo su konteineriais metodus*

### 1.10.3 Trečiosios strategijos išvados

1000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 0.0321626              |
| Listas            | 0.02778266             |
| Dekas             | 0.03375468             |

10000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 0.10179046             |
| Listas            | 0.08829786             |
| Dekas             | 0.148642               |

100000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 0.5735542              |
| Listas            | 0.543196               |
| Dekas             | 1.051894               |

1000000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 5.157186               |
| Listas            | 4.589734               |
| Dekas             | 10.72832               |

10000000

| Konteinerio tipas | Vid. laikas sekundėmis |
|-------------------|------------------------|
| Vektorius         | 54.86644               |
| Listas            | 54.11488               |
| Dekas             | -                      |

#### 1.10.4 Visų strategijų išvados

1000

| Konteinerio tipas | Pirma strategija | Antra strategija | Trečia strategija |
|-------------------|------------------|------------------|-------------------|
| Vektorius         | 0.06018572       | 0.05090222       | 0.0321626         |
| Listas            | 0.038556         | 0.1495786        | 0.02778266        |
| Dekas             | 0.05684944       | 0.04837712       | 0.03375468        |

10000

| Konteinerio tipas | Pirma strategija | Antra strategija | Trečia strategija |
|-------------------|------------------|------------------|-------------------|
| Vektorius         | 0.10010296       | 0.07974518       | 0.10179046        |
| Listas            | 0.1621798        | 0.1139308        | 0.08829786        |
| Dekas             | 0.3532868        | 0.2323468        | 0.148642          |

100000

| Konteinerio tipas | Pirma strategija | Antra strategija | Trečia strategija |
|-------------------|------------------|------------------|-------------------|
| Vektorius         | 0.515036         | 0.4810722        | 0.5735542         |
| Listas            | 1.01146          | 0.5868968        | 0.543196          |
| Dekas             | 2.421398         | 1.3812           | 1.051894          |

1000000

| Konteinerio tipas | Pirma strategija | Antra strategija | Trečia strategija |
|-------------------|------------------|------------------|-------------------|
| Vektorius         | 5.244066         | 4.902806         | 5.157186          |
| Listas            | 6.887438         | 6.103116         | 4.589734          |
| Dekas             | 22.9448          | 10.78286         | 10.72832          |

10000000

| Konteinerio tipas | Pirma strategija | Antra strategija | Trečia strategija |
|-------------------|------------------|------------------|-------------------|
| Vektorius         | 54.68584         | 57.99392         | 54.86644          |
| Listas            | 92.91066         | 68.92654         | 54.11488          |
| Dekas             | 1791.95          | 1270.3142        | -                 |

#### 1.10.5 Kompiuterio komponentai

- OS: GNU/Linux
- CPU: Intel i7-7500U
- GPU: Intel HD Graphics 620

- RAM: 8GB



## Chapter 2

# Hierarchical Index

### 2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                       |    |
|-----------------------|----|
| Human . . . . .       | 23 |
| Student1 . . . . .    | 24 |
| Vector< T > . . . . . | 25 |



## Chapter 3

# Class Index

### 3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                   |    |
|-----------------------------------|----|
| <a href="#">Human</a>             | 23 |
| <a href="#">Student1</a>          | 24 |
| <a href="#">Vector&lt; T &gt;</a> | 25 |





## Chapter 4

# File Index

### 4.1 File List

Here is a list of all documented files with brief descriptions:

|                                                                      |    |
|----------------------------------------------------------------------|----|
| /home/xxmp/GitHub/OP2/program/ <a href="#">header.h</a> . . . . .    | 27 |
| /home/xxmp/GitHub/OP2/program/ <a href="#">human.hpp</a> . . . . .   | 27 |
| /home/xxmp/GitHub/OP2/program/ <a href="#">student.hpp</a> . . . . . | 28 |
| /home/xxmp/GitHub/OP2/program/ <a href="#">vector.hpp</a> . . . . .  | 31 |

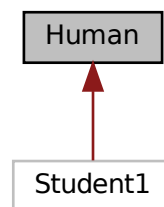


## Chapter 5

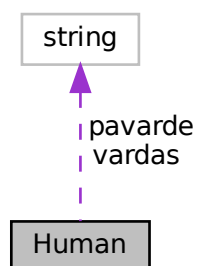
# Class Documentation

### 5.1 Human Class Reference

Inheritance diagram for Human:



Collaboration diagram for Human:



### Public Member Functions

- **Human** (const std::string &v, const std::string &p)
- **Human** & **operator=** (const **Human** &other)
- virtual void **setVardas** (std::string vardas\_)=0

- virtual void **setPavarde** (std::string pavarde\_)=0
- virtual std::string **getVardas** () const =0
- virtual std::string **getPavarde** () const =0

### Protected Attributes

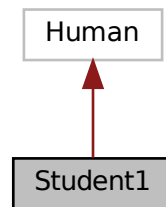
- std::string **vardas**
- std::string **pavarde**

The documentation for this class was generated from the following file:

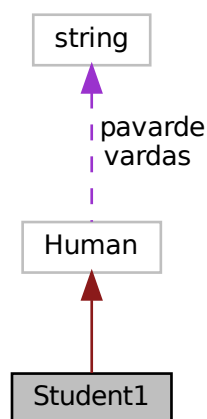
- /home/xxmp/GitHub/OP2/program/human.hpp

## 5.2 Student1 Class Reference

Inheritance diagram for Student1:



Collaboration diagram for Student1:



### Public Member Functions

- **Student1** (string v="Default", string p="Default", int e=0, vector< int > m={0})

- **Student1** (const [Student1](#) &other)
- [Student1](#) & **operator=** (const [Student1](#) &other)
- **Student1** ([Student1](#) &&other) noexcept
- [Student1](#) & **operator=** ([Student1](#) &&other) noexcept
- bool **operator==** (const [Student1](#) &other) const
- void **setVardas** (string vardas\_) override
- void **setPavarde** (string pavarde\_) override
- void **setEgzaminas** (int egzaminas\_)
- void **setMark** (vector< int > mark\_)
- string [getVardas](#) () const override
- string [getPavarde](#) () const override
- int **getEgzaminas** () const
- vector< int > **getMark** () const
- float **getMedian** () const
- float **getResult** () const
- void **GenerateNames** ()

## Friends

- std::ostream & **operator<<** (std::ostream &os, const [Student1](#) &stud)
- std::ifstream & **operator>>** (std::ifstream &in, [Student1](#) &stud)
- std::istream & **operator>>** (std::istream &is, [Student1](#) &stud)

## 5.2.1 Member Function Documentation

### 5.2.1.1 getPavarde()

string Student1::getPavarde ( ) const [inline], [override], [virtual]  
Implements [Human](#).

### 5.2.1.2 getVardas()

string Student1::getVardas ( ) const [inline], [override], [virtual]  
Implements [Human](#).

The documentation for this class was generated from the following files:

- /home/xxmp/GitHub/OP2/program/student.hpp
- /home/xxmp/GitHub/OP2/program/student.cpp

## 5.3 Vector< T > Class Template Reference

### Public Member Functions

- **Vector** (size\_t initial\_capacity)
- **Vector** (std::initializer\_list< T > list)
- **Vector** ([Vector](#) &&other) noexcept
- **Vector** (const [Vector](#) &other)
- [Vector](#) & **operator=** (const [Vector](#) &other)
- [Vector](#) & **operator=** ([Vector](#) &&other)
- template<typename input >  
**Vector** (input first, input last)
- T & **at** (size\_t index)
- T & **operator[]** (size\_t index)
- T & **front** ()

- `T & back ()`
- `T * begin ()`
- `T * end ()`
- `T * rbegin ()`
- `T * rend ()`
- `size_t get_size () const`
- `size_t get_capacity () const`
- `bool empty () const`
- `void reserve (size_t new_capacity)`
- `void shrink_to_fit ()`
- `size_t max_size () const`
- `void clear ()`
- `void insert (size_t index, const T &value)`
- `void insert (size_t index, size_t count, const T &value)`
- `void push_back (const T &value)`
- `void pop_back ()`
- `void erase (size_t index)`
- `T * erase (T *first, T *last)`

The documentation for this class was generated from the following file:

- `/home/xxmp/GitHub/OP2/program/vector.hpp`

## Chapter 6

# File Documentation

### 6.1 header.h

```
1 #include <algorithm>
2 #include <cctype>
3 #include <cstdio>
4 #include <cstdlib>
5 #include <iomanip>
6 #include <iostream>
7 #include <random>
8 #include <sstream>
9 #include <string>
10 #include <fstream>
11 #include <vector>
12 #include <chrono>
13 #include <filesystem>
14 #include <ios>
15 #include <stdexcept>
16
17 using std::cout;
18 using std::cin;
19 using std::endl;
20 using std::setw;
21 using std::string;
22 using std::vector;
23 using std::ifstream;
24 using std::cerr;
25 using std::left;
26 using std::right;
27
28 bool isValid ( string number );
29
30 bool isValid ( int number );
31
32 int randomNumber (int a, int b);
33
34 void readFile(vector<string>& name);
35
36 string get_name(vector <string> name);
37
38 //void count_marks (vector<Student>& stud);
39
40 vector<string> listTxtFiles();
41
42 void generate_file (int number);
43
44
```

### 6.2 human.hpp

```
1 #include <string>
2
3 class Human {
4     protected:
5
6         std::string vardas;
7         std::string pavarde;
8
9     public:
10
11         // Default constructor
12         Human() : vardas(""), pavarde("") {}

```

```

13
14 // Parameterized constructor
15 Human(const std::string& v, const std::string& p) : vardas(v), pavarde(p) {}
16
17
18 // virtualus destruktorius
19 virtual ~Human() {
20     //std::cout << "Virtual destructor Human was called" << std::endl;
21 }
22
23 // Copy assignment operator
24 Human& operator=(const Human& other) {
25     if (this != &other) {
26         vardas = other.vardas;
27         pavarde = other.pavarde;
28     }
29     return *this;
30 }
31
32 //virtualus setteriai
33 virtual void setVardas(std::string vardas_) = 0;
34
35 virtual void setPavarde(std::string pavarde_) = 0;
36
37 //virtualus getteriai
38 virtual std::string getVardas() const = 0;
39
40 virtual std::string getPavarde() const = 0;
41 };

```

## 6.3 student.hpp

```

1 #include <iomanip>
2 #include <ios>
3 #include <istream>
4 #include <ostream>
5 #include <fstream>
6 #include <sstream>
7 #include <string>
8 #include <vector>
9 #include <iostream>
10 #include <chrono>
11 #include <algorithm>
12 #include "../program/human.hpp"
13 #include "vector.hpp"
14
15 using std::string;
16 using std::vector;
17
18
19 class Student1 : private Human {
20
21     private:
22         //string vardas;
23         //string pavarde;
24         int egzaminas;
25         vector<int> mark;
26     public:
27
28         // konstruktorius
29         Student1(string v = "Default", string p = "Default", int e = 0, vector<int> m = {0}) :
            egzaminas(e), mark(m) {
30             //std::cout << "Iskviestas Konstruktorius studentui " << v << std::endl;
31             vardas = v;
32             pavarde = p;
33         }
34
35         // destruktorius
36         ~Student1() {
37             //std::cout << "Iskviestas Destruktorius studentui " << vardas << std::endl;
38             mark.clear();
39         }
40
41         // copy konstruktorius
42         //Student1 (const Student1 &other) : Student1(other.vardas, other.pavarde, other.egzaminas,
            other.mark) { // kodėl jei čia parašau : Student1 vardas = other.vardas....
43             Student1 (const Student1 &other) { // kodėl jei čia parašau : Student1 vardas = other.vardas....
44                 vardas = other.vardas;
45                 pavarde = other.pavarde;
46                 egzaminas = other.egzaminas;
47                 mark = other.mark;
48                 //std::cout << "Iskviestas Copy Konstruktorius studentui " << vardas << std::endl;
49             }
50
51         // copy assignment

```



```

52     Student1& operator=(const Student1 &other) {
53         if (this != &other) { // Patikrinti ar ne self-assigninama
54             //std::cout << "Assigning vardas, this: " << this << ", other: " << &other << std::endl;
55             vardas = other.vardas;
56             //std::cout << "Assigning pavarde" << std::endl;
57             pavarde = other.pavarde;
58             //std::cout << "Assigning egzaminas" << std::endl;
59             egzaminas = other.egzaminas;
60             //std::cout << "Assigning mark" << std::endl;
61             mark = other.mark;
62         }
63         else std::cout << "Self-assignment, skipping" << std::endl;
64         //std::cout << "Iskviestas Copy Assignmentas studentui " << vardas << std::endl;
65         return *this;
66     }
67
68     //move konstruktorius
69     Student1(Student1&& other) noexcept : Student1(other.vardas ,other.pavarde, other.egzaminas,
other.mark) {
70         //other = Student1();
71         //std::cout << "Iskviestas Move Konstruktorius studentui " << vardas << std::endl;
72         //}
73
74     Student1(Student1&& other) noexcept :
75     Human(std::move(other.vardas), std::move(other.pavarde)), // Move base class members
76         egzaminas(other.egzaminas),
77         mark(std::move(other.mark)) {
78         // Reset the moved-from object
79         other.vardas = "Default";
80         other.pavarde = "Default";
81         other.egzaminas = 0;
82         other.mark = {0};
83         //std::cout << "Iskviestas Move Konstruktorius studentui " << vardas << std::endl;
84     }
85
86     // move assignment
87     /*
88     Student1& operator=(Student1&& other) noexcept {
89         if (this != &other) { // Patikrinti ar ne self-moving
90             vardas = other.vardas;
91             pavarde = other.pavarde;
92             egzaminas = other.egzaminas;
93             mark = other.mark;
94
95             other = Student1();
96         }
97         else std::cout << "Self-moving, skipping" << std::endl;
98         //std::cout << "Iskviestas Move Assignmentas studentui " << vardas << std::endl;
99         return *this;
100     }
101     */
102
103     Student1& operator=(Student1&& other) noexcept {
104         if (this != &other) { // Avoid self-move
105             // Move base class members
106             Human::operator=(std::move(other));
107
108             // Move member variables
109             egzaminas = other.egzaminas;
110             mark = std::move(other.mark);
111
112             // Reset the moved-from object
113             other.vardas = "Default";
114             other.pavarde = "Default";
115             other.egzaminas = 0;
116             other.mark = {0};
117
118         }
119         //std::cout << "Iskviestas Move Assignmentas studentui " << vardas << std::endl;
120         return *this;
121     }
122
123     // Equality operator
124     bool operator==(const Student1& other) const {
125         return (vardas == other.vardas &&
126             pavarde == other.pavarde &&
127             egzaminas == other.egzaminas &&
128             mark == other.mark);
129     }
130
131     // setteriai
132
133     void setVardas (string vardas_) override;
134
135     void setPavarde(string pavarde_) override;
136
137     void setEgzaminas(int egzaminas_);

```

```

138
139 void setMark(vector<int> mark_);
140
141 // getteriai
142
143 string getVardas()const override {
144     return vardas;
145 }
146
147 string getPavarde()const override {
148     return pavarde;
149 }
150
151 int getEgzaminas()const {
152     return egzaminas;
153 }
154
155 vector<int> getMark()const {
156     return mark;
157 }
158
159 friend std::ostream& operator<<(std::ostream& os, const Student1& stud) {
160     int g = 20;
161     os << std::setw(g) << std::left << stud.getVardas()
162     << std::setw(g) << std::left << stud.getPavarde()
163     << std::setw(g) << std::left << std::setprecision(3)
164     << stud.getResult() << std::setw(g) << std::left
165     << std::setprecision(3) << stud.getMedian() << std::endl;
166     return os;
167 }
168
169 // ifstreama dar padaryt
170 friend std::istream& operator>>(std::istream& in, Student1& stud) {
171     std::string temp;
172     std::getline(in, temp); // Read the entire line
173     //std::cout << "nuskaitoma eilute \n" << std::endl;
174
175     if (temp.empty()) return in; // Skip empty lines
176
177     string random;
178     std::istringstream stream(temp); // Convert the line into a string stream
179     //std::cout << "konvertuojama i string stream \n" << std::endl;
180
181     stream >> stud.vardas >> stud.pavarde;
182
183     //std::cout << "ivesta vardas ir pavarde" << std::endl;
184     //std::cout << stud.getPavarde() << " " << stud.getPavarde() << "\n";
185
186     int marks;
187     stud.mark.clear(); // Clear marks vector
188     while (stream >> marks) {
189         stud.mark.push_back(marks);
190         //std::cout << "ivestas pazymys \n" << std::endl;
191     }
192
193     if (!stud.mark.empty()) {
194         stud.egzaminas = stud.mark.back();
195         stud.mark.pop_back();
196     } else {
197         stud.egzaminas = 0; // Default value if no marks are provided
198     }
199     //std::cout << "ivestas egzaminas \n" << std::endl;
200
201
202
203     return in;
204 }
205
206
207 friend std::istream& operator>>(std::istream& is, Student1& stud) {
208     std::string egzaminas, mark_temp;
209     vector<int> mark {};
210
211     std::cout << "Enter name: ";
212     is >> stud.vardas;
213
214     std::cout << "Enter surname: ";
215     is >> stud.pavarde;
216
217     while (true) {
218         std::cout << "Enter exam mark (0-10): ";
219         std::cin >> egzaminas;
220         if (std::stoi(egzaminas) >= 0 && std::stoi(egzaminas) <= 10) {
221             stud.egzaminas = std::stoi(egzaminas);
222             break;
223         }
224     }

```

```

225
226     while (true) {
227         std::cout << "Enter a mark (or 'q' to quit): ";
228         std::cin >> mark_temp;
229
230         if (mark_temp == "q") {
231             stud.mark = mark;
232             mark.clear();
233             system("clear");
234             break;
235         }
236
237         if (std::stoi(mark_temp) >= 0 && std::stoi(mark_temp) <= 10) {
238             mark.push_back(std::stoi(mark_temp));
239         }
240     }
241
242     return is;
243 }
244
245 float getMedian() const;
246
247 float getResult() const;
248
249 void GenerateNames();
250
251 };
252
253 void print(vector<Student1> &stud);
254
255 void import_file (Vector<Student1> &stud, string filename);
256
257 void sort_students (Vector<Student1>& stud);
258
259 void choose_printing_method (Vector<Student1>& stud);
260
261 void print_marks (Vector<Student1> stud);
262
263 void write_marks (Vector<Student1> stud, string name);
264
265 void divide_file1 (Vector<Student1>& stud, Vector<Student1>& kietiakai, Vector<Student1>& nuskriaustukai,
    string filename);
266
267 void divide_file2 (Vector<Student1>& stud, Vector<Student1>& nuskriaustukai, string filename);
268
269 void divide_file3 (Vector<Student1>& stud, Vector<Student1>& nuskriaustukai, string filename);
270
271 void pagrindinis_divide (Vector<Student1>& stud, Vector<Student1>& kietiakai, Vector<Student1>&
    nuskriaustukai, int num);
272
273 void pagrindinis_divide_choice(Vector<Student1>& stud);
274
275 void sort_file (Vector<Student1>& stud, string name);
276
277 void rule_of_five();
278
279 void print_metrics (string filename, float data, int num, int strategija);
280
281 int lineCount (string filename);
282
283 string extractNumbers(const std::string& str);
284
285 void compare_vectors();

```

## 6.4 vector.hpp

```

1 #include <cstdint>
2 #include <iostream>
3 #include <limits>
4
5 template <typename T>
6 class Vector {
7     private:
8         T* data;
9         size_t size;
10        size_t capacity;
11
12        void resize(size_t new_size) {
13            if (new_size <= size) {
14                return; // Tik sumazinti dydi aij bbz kas cia zodziu palikt vietoj
15            }
16            else {
17                reserve(new_size); // Rezervuoja vieta jeigu reikia
18                for (size_t i = size; i < new_size; ++i)
19                    data[i] = T(); // Naudoja default konstruktoriu naujiems objektams

```

```

20         size = new_size;
21     }
22 }
23
24 public:
25     //-----Member-funkcijos-----//
26
27     // Konstruktorius
28     Vector() : data(nullptr), size(0), capacity(0) {}//std::cout << "Iskviestas konstruktorius" <<
std::endl;}
29
30     // Konstruktorius su nustatytu dydziu
31     explicit Vector(size_t initial_capacity) : data(new T[initial_capacity]), size(0),
capacity(initial_capacity) {}//std::cout << "Iskviestas konstruktorius" << std::endl;}
32
33     // Konstruktorius su aibe
34     Vector(std::initializer_list<T> list) : size(list.size()) {
35         data = new T[size];
36         size_t i = 0;
37         for (const auto& elem : list) {
38             data[i++] = elem;
39         }
40     }
41
42     // Move konstruktorius
43     Vector(Vector&& other) noexcept
44     : data(other.data), size(other.size), capacity(other.capacity) {
45         other.data = nullptr;
46         other.size = 0;
47         other.capacity = 0;
48     }
49
50     // Copy konstruktorius
51     Vector(const Vector& other) : size(other.size), capacity(other.capacity) {
52         data = new T[capacity];
53         for (size_t i = 0; i < size; ++i) {
54             data[i] = other.data[i];
55         }
56     }
57
58     // Destruktorius
59     ~Vector() {
60         delete[] data;
61         //std::cout << "Iskviestas destruktoriaus" << std::endl;
62     }
63
64     // Copy assignment
65     // operatorius= pvz b = a
66     Vector& operator=(const Vector& other) {
67         if (this != &other) {
68             delete[] data;
69             size = other.size;
70             capacity = other.capacity;
71             data = new T[capacity];
72             for (size_t i = 0; i < size; ++i) {
73                 data[i] = other.data[i];
74             }
75         }
76     }
77     return *this;
78 }
79
80     // Move assignment
81     // operatorius= pvz b = std::move(a)
82     Vector& operator=(Vector&& other) {
83         if (this != &other) {
84             delete[] data;
85
86             data = other.data;
87             size = other.size;
88             capacity = other.capacity;
89
90             other.data = nullptr;
91             other.size = 0;
92             other.capacity = 0;
93         }
94     }
95     return *this;
96 }
97
98     // Konstruktorius is dviejų operatorių ar pointerių
99     template<typename input>
100     Vector(input first, input last) {
101         size = std::distance(first, last);
102         capacity = size;
103         data = new T[capacity];

```

```

105         size_t i = 0;
106         for (auto it = first; it != last; ++it, ++i) {
107             data[i] = *it;
108         }
109     }
110
111     //-----ELEMENT-ACCESS-----//
112
113     // Pasiukti elementa su patikrinimu
114     T& at(size_t index) {
115         if (index >= size) {
116             throw std::out_of_range("Index out of range");
117         }
118         return data[index];
119     }
120
121     // Pasiukti elementa per indeksa
122     T& operator[](size_t index) {
123         return data[index];
124     }
125
126     // Paimti pirma elementa
127     T& front() {
128         return data[0];
129     }
130
131     // Pasiukti paskutini elementa
132     T& back() {
133         return data[size-1];
134     }
135
136     //-----ITERATORS-----//
137
138     // Pradzia
139     T* begin() {
140         return data;
141     }
142
143     // Pabaiga
144     T* end() {
145         return data+size;
146     }
147
148     // R Pradzia
149     T* rbegin() {
150         return data+size-1;
151     }
152
153     // R Pabaiga
154     T* rend() {
155         return data-1;
156     }
157
158     //-----CAPACITY-----//
159
160     // Gauti size
161     size_t get_size()const {
162         return size;
163     }
164
165     // Gauti capacity
166     size_t get_capacity()const {
167         return capacity;
168     }
169
170     // Patikrinti ar vektorius tuscias
171     bool empty()const {
172         if (size == 0) return true;
173         else return false;
174     }
175
176     // Rezervuoti vieta atmintyje
177     void reserve(size_t new_capacity) {
178         if (new_capacity <= capacity) return;
179         // Grow at least double to avoid frequent allocations
180         size_t new_cap = (capacity == 0) ? 1 : capacity;
181         while (new_cap < new_capacity) new_cap *= 2;
182         T* new_data = new T[new_cap];
183         for (size_t i = 0; i < size; ++i)
184             new_data[i] = data[i];
185         delete[] data;
186         data = new_data;
187         capacity = new_cap;
188     }
189
190     // Susrinkinti capacity
191     void shrink_to_fit() {

```

```

192         if (capacity > size) {
193             resize(size);
194         }
195     }
196
197     // Max dydis
198     size_t max_size() const {
199         return std::numeric_limits<size_t>::max() / sizeof(T);
200     }
201
202     //-----Modifiers-----//
203
204     // Išvalyti vektoriu
205     void clear() {/?
206         //delete[] data;
207         //data = nullptr;
208         size = 0;
209         //capacity = 0;
210     }
211
212     // Idėti elementą x į y vietą
213     void insert(size_t index, const T& value) {
214         if (index > size) {
215             throw std::out_of_range("Insert index out of range");
216         }
217
218         if (size == capacity) {
219             reserve(capacity == 0 ? 1 : capacity * 2);
220         }
221
222         for (size_t i = size; i > index; --i) {
223             data[i] = data[i - 1];
224         }
225         data[index] = value;
226         ++size;
227     }
228
229     // Idėti x kopiją y elemento prieš z vietą
230     void insert(size_t index, size_t count, const T& value) {
231         if (index > size) {
232             throw std::out_of_range("Insert index out of range");
233         }
234
235         if (size == capacity) {
236             reserve(capacity == 0 ? 1 : capacity * 2);
237         }
238
239         // Visus egzistuojančius elementus perkelti į dešinę
240         for (size_t i = size; i > index; --i) {
241             data[i + count - 1] = data[i - 1];
242         }
243         // Į senas vietas įdėti naujus elementus
244         for (size_t i = 0; i < count; ++i) {
245             data[index + i] = value;
246         }
247         size += count;
248     }
249
250     // Pridėti elementą į galą
251     void push_back(const T& value) {
252         if (size == capacity) {
253             reserve(capacity == 0 ? 1 : capacity * 2);
254         }
255         data[size++] = value;
256         //size++;
257     }
258
259     // Ištrinti elementą iš galo
260     void pop_back() {
261         if (size == 0) {
262             throw std::out_of_range("Index out of range");
263         }
264         data[size-1].~T(); // Iškviešti destruktorių paskutiniam elementui(?)
265         --size;
266     }
267
268
269     // Ištrinti elementą iš pasirinktos vietos
270     void erase(size_t index) {
271         if (index >= size) {
272             throw std::out_of_range("Index out of range");
273         }
274         for (size_t i = index; i < size - 1; ++i) {
275             data[i] = data[i+1];
276         }
277
278         --size;

```

```
279     }
280
281     // Istringinti elementu aibe is pasirinktos vietas
282     T*erase(T* first, T* last) {
283         //
284         //
285         //std::cout << "Erase from index: " << (first - data) << " to index: " << (last - data) << "
(size=" << size << ")" << std::endl;
286         //
287         // Bounds check (optional, but for debugging it's helpful)
288         if (first < data || last > data + size || first > last) {
289             throw std::out_of_range("Invalid erase range");
290         }
291
292         size_t erase_count = last - first;
293         if (erase_count == 0) return first;
294
295         // Move trailing elements forward
296         for (T* it = first; last != data + size; ++it, ++last) {
297             *it = std::move(*last);
298         }
299
300         size -= erase_count;
301         return first;
302     }
303
304     //-----//
305
306     };
```





# Index

[/home/xxmp/GitHub/OP2/program/header.h](#), [27](#)  
[/home/xxmp/GitHub/OP2/program/human.hpp](#), [27](#)  
[/home/xxmp/GitHub/OP2/program/student.hpp](#), [28](#)  
[/home/xxmp/GitHub/OP2/program/vector.hpp](#), [31](#)

`getPavarde`  
    Student1, [25](#)  
`getVardas`  
    Student1, [25](#)

Human, [23](#)

Student1, [24](#)  
    getPavarde, [25](#)  
    getVardas, [25](#)

Vector< T >, [25](#)